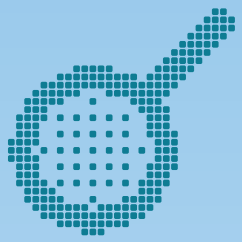

QSOE

Quick and Secure Operating Environment

Application Porting Guide

Bringing existing software to QSOE

Document:	`QSOE-APG`
Revision:	Rev B — Draft
Applies to:	QSOE 0.2

Yuri Zaporozhets

September 2026

Contents

Chapter 1

Introduction

This guide is for someone with an existing program — a Unix or POSIX utility, a library, a piece of QNX software — who wants to build and run it on QSOE. It explains what QSOE gives you to build against, the ways it differs from a conventional Unix that a port must account for, and how to find out whether the functions your code depends on are actually there yet.

1.1 Who this guide is for

You should read this guide if you are bringing *outside* code to QSOE. If instead you are writing new software specifically for QSOE — a resource server, a driver, a program built around QSOE's own message-passing API — the *Programming Book* (`QSOE-PRG`) is the better starting point. The division is simple: the Programming Book is about building *with* what QSOE provides; this guide is about reconciling code written for *another* system with what QSOE provides.

A working knowledge of C and of the POSIX/Unix programming model is assumed. Familiarity with QNX is helpful where this guide draws comparisons, but not required.

1.2 What "porting" means on QSOE

Porting to QSOE is **source-level** work: you recompile your program from source against QSOE's headers and libraries. It is *not* a matter of running an existing binary.

This is a deliberate consequence of how QSOE is built. There is no fixed binary interface to target: QSOE runs on two different microkernels (the *Design & Architecture* guide describes the two-kernel model), and a program is a single dynamically linked binary that binds the system's `libc.so` at load time. Even a QNX program cannot be run as-is — the only QNX/RISC-V system was a separate project with its own ABI, and QSOE shares none of it. What carries across is the *source*: well-written portable C, recompiled.

The practical upshot, returned to throughout this guide:

- You will **recompile**, not relink against a prebuilt binary or drop in a foreign executable.
- Your program will be **dynamically linked** against the system libc — static linking of libc is not how QSOE works (Chapter 2).
- The port succeeds to the extent your code is **portable C** that leans on POSIX rather than on one system's private extensions.

1.3 The compatibility goal

QSOE's long-term aim is concrete: by version 1.0, the QSOE C library should be compatible enough to **recompile and run QNX userland software with no changes beyond those**

QSOE's own design forces. That qualifier is not a hedge — it is the honest bar. "Unmodified" cannot be taken literally, because QSOE deliberately omits a few facilities for good (no `fork()`, no `select()`, no `brk()`; §1.4): code that uses them must be adapted, and no amount of library work will change that. Short of those deliberate differences, the goal is that QNX source recompiles and runs as written. This is source and recompile compatibility — there is no QNX/RISC-V ABI to preserve — not binary compatibility.

Version 0.1, which this guide describes, is a long way before that bar. The foundation is in place — a dynamically linked userspace, a Korn shell, the POSIX process and IPC model — but the C library is young, and a great many standard functions are not implemented yet. This guide is therefore as much about *recognizing what is not there* as about the mechanics of building: porting a non-trivial program to 0.1 means meeting gaps, and the goal here is to let you find them quickly and decide what to do about each.

1.4 What makes QSOE different to port to

Two distinct things separate QSOE from a mature Unix, and this guide keeps them separate because they call for different responses:

1. **Deliberate structural choices** that will never change — no `fork()`, no `select()`, no `brk()`. These are permanent design decisions, and code that relies on them must be adapted to the QSOE way (Chapter 3). The adaptation is usually small and mechanical.
2. **An incomplete C library** — functions that are simply not written yet, but are not excluded in principle and arrive release by release (0.2 added a little over a hundred of them) (Chapter 4 and Appendix A). Here the response is to check, and then to avoid, shim, wait, or contribute.

Keeping these apart matters: a missing `fork()` is a fact to design around for good, while a missing `glob()` is a gap that may close next release. Confusing the two leads to either needless rewrites or false hopes.

1.5 How to read this guide

- Chapter 2 — the build environment: the toolchain, the dynamic-linking requirement, and how to compile and link a program.
- Chapter 3 — the permanent structural differences and the QSOE idiom to use instead of each.
- Chapter 4 — the state of the C library: how to tell what exists, what is missing by category, and the strategies for coping.
- Chapter 5 — the runtime environment a ported program meets: the filesystem, the namespace, and the console.
- Chapter 6 — porting in practice: a checklist and a worked example.
- Appendix A — the inventory of standard-library functions missing in QSOE 0.1.

Chapter 2

The Build Environment

Before any source-level differences, a port has to *build*. This chapter describes the environment you build in: a cross-compiler on a Linux host, a header world that contains only QSOE's own headers, and a strict dynamic-linking contract that every QSOE program obeys. Understanding these three up front saves a great deal of confusion later, because each rejects things a program built for a self-hosted Unix takes for granted.

2.1 A cross-compiled system

QSOE 0.1 is **cross-compiled**. You build on a Linux host with a RISC-V cross-toolchain and run the result on QSOE; there is no on-target compiler yet. A port's build therefore runs on your development machine, not on QSOE.

Planned content. The "yet" is deliberate: a native C compiler is planned for QSOE — **qjmcc**, the QSOE C compiler, derived from `pcc` (the Portable C Compiler of Johnson and Magnusson), which already emits RISC-V code. It will let you build software on QSOE itself rather than only cross-compiling from a host. It is a natural fit: like QSOE, qjmcc is C-only and BSD-licensed. No release is fixed for it yet.

The toolchain is the standard GNU RISC-V cross-toolchain, invoked through the `riscv64-linux-gnu-` prefix (`riscv64-linux-gnu-gcc` and friends). Two ABI facts are fixed and must match everywhere: the **architecture** is rv64gc (64-bit RISC-V with the general-purpose and compressed extensions), and the **ABI** is `lp64d`, the hard-float calling convention. In full, the architecture flags are:

```
-march=rv64imafdc_zicsr_zifencei_zicntr -mabi=lp64d -mcmodel=medany
```

The C dialect is **strict C11** (`-std=c11`, not `gnu11`). Code that relies on GNU dialect features in their bare forms will need the underscore-prefixed spellings (`__asm__`, `__typeof__`); this is rarely an issue for portable code but occasionally surfaces in software that assumed a GNU compiler.

2.2 A freestanding header world

The single most common surprise when porting is the headers. QSOE programs are compiled `-nostdinc`: the host's system headers are *not* on the include path. Only two header sources are:

- QSOE's own C-library headers (the `libc/include` tree), supplied with `-isystem`; and
- the compiler's own freestanding headers (`stddef.h`, `stdint.h`, `stdarg.h`, and the like).

In other words, a function or type is available to your code only if a *QSOE* header declares it — never because the host's glibc happens to. This is what makes the gaps of Chapter 4 concrete at compile time: if QSOE does not provide `glob()`, there is no `glob.h` to include, and the reference does not compile. That is a feature, not a frustration — it means a build either has everything it needs or fails honestly, with no silent dependence on a header that will not be there at runtime.

The environment is also **freestanding** (`-ffreestanding`) and declares an X/Open feature level of 700 (`-D_XOPEN_SOURCE=700`), which is the POSIX.1-2008 / SUSv4 baseline the headers expose.

2.3 The dynamic-linking contract

Every QSOE program is **dynamically linked against the system `libc.so`**. There is no static-libc path, by design: because the same binary must run on either kernel and only the system's `libc.so` carries the right per-kernel seam, a program cannot bake libc into itself (the *Design & Architecture* guide explains why). For a port this means a few concrete things:

- You link, but do not embed, the C library: your program carries a `DT_NEEDED` reference to `libc.so`, which `taskman` resolves when it loads the program. There is no interpreter to name: a QSOE binary carries no `PT_INTERP` header, and link with `-Wl,-no-dynamic-linker` so that yours does not either.
- You link the shared C runtime start-up, `crt0.o`, which sits beside `libc.so`. It is OS-independent; the per-kernel bring-up lives inside `libc.so`.
- Binding is **eager**: every symbol is resolved before the program runs (`-Wl,-z,now`), never on first call. This is a real-time requirement, and it has a porting consequence worth knowing — an unresolved symbol is reported *at load time*, by name, rather than surfacing only when some rarely-taken path first calls it.

A practical corollary of eager binding and the freestanding headers: a missing function tends to announce itself early — either the compile fails (no declaration) or the load fails (no symbol) — rather than lurking. When you do hit an unresolved symbol at load, its name is exactly the function you need to account for (Chapter 4).

2.4 Building a program the QSOE way

The path of least resistance is to build your program *inside* the shared userspace tree (`quser`), as a component, and let the shared build rules apply all of the above for you. A component is a directory with your sources and a small Makefile; the shared `build_prog.mk` supplies the toolchain, the flags, the headers, `crt0.o`, and the link against `libc.so`. A complete Makefile for a single-program port is as short as this:

```
include ../../common.mk

PROGRAM := mytool         # produces mytool.elf
SRCS    := main.c parse.c

# Optional knobs when you need them:
#   EXTRA_CFLAGS  := -DFOO
#   EXTRA_LDLIBS  := -lfoo
include $(QUSER)/build_prog.mk
```

That is the recommended way: it guarantees your binary matches the system's ABI, header set, and link contract exactly, because it is built the same way as every other QSOE program. Building outside the tree is possible — the flags in this chapter are the whole contract — but then keeping them in step with the system is your responsibility, and the component path simply avoids that.

2.5 C, not C++

QSOE is a **C-only** environment. There is no C++ runtime, no C++ standard library, and the headers carry no `extern "C"` guards. Software written in C++ is out of scope for porting to 0.1; a C++ program must be reduced to C, or wait for support that is not on the near horizon. Plain C that merely passed through a C++ compiler elsewhere is fine once compiled as C.

Chapter 3

Structural Differences — What to Use Instead

This chapter covers the differences a port must adapt to *for good*. Unlike the library gaps of the next chapter, none of these will be filled in a later release — they are deliberate design choices (the *Design & Architecture* guide explains the reasoning). The good news is that each has a direct QSOE replacement, and the adaptation is usually small and mechanical. The work is recognizing the pattern in your code and substituting the QSOE one.

3.1 Process creation: `posix_spawn` instead of `fork`

QSOE has no `fork()`. A new process is created in one step, with `posix_spawn()` (and `posix_spawnp()`), which QSOE provides including file actions and spawn attributes.

The overwhelmingly common use of `fork()` — immediately followed by an `exec` to run another program — maps directly onto `posix_spawn`, which *is* that fork-then-exec sequence as a single call. The translation is mechanical:

```
/* The classic fork + exec idiom ... */
pid_t pid = fork();
if (pid == 0) {
    /* child: optionally tweak fds, then exec */
    dup2(fd, STDOUT_FILENO);
    execvp(path, argv);
    _exit(127);
}
/* parent continues, later waitpid(pid, ...) */

/* ... becomes one posix_spawn call: */
posix_spawn_file_actions_t fa;
posix_spawn_file_actions_init(&fa);
posix_spawn_file_actions_adddup2(&fa, fd, STDOUT_FILENO);
posix_spawn(&pid, path, &fa, NULL, argv, environ);
/* parent continues, later waitpid(pid, ...) */
```

The file actions cover what the child used to do between `fork` and `exec`: redirecting descriptors, closing them, opening files. Anything you genuinely cannot express as a file action belongs in a small helper program you `spawn` instead.

The `exec*` family too. Standalone `exec` — replacing the running program's image in place, without a preceding `fork` — is not provided either, and for the same reason: process creation and image replacement are both `posix_spawn`'s job on QSOE. `exec` is in fact a poor fit beyond

redundancy: in a multithreaded program it discards every thread but the caller and resumes single-threaded, which sits badly with a system where every process also runs a system thread. Where you would have `exec`'d, `spawn` the new program and exit; the only visible difference is that the new program gets a new pid.

What does *not* translate is `fork()` used *without* a following `exec` — a program that forks to get a second copy of itself sharing the parent's memory and code path (a common pre-threads concurrency idiom). QSOE has no such facility: a spawned process is a fresh, independent program from a path, not a clone of the caller (§3.4). Code of that shape must be redesigned — typically around threads (`pthread_create`) for shared-memory concurrency, or around an explicit helper program plus message passing for separate processes. This is the one structural change that can be more than mechanical, and it is worth identifying early in a port.

Staying resident: `procmgr_detach` instead of the double fork

One `fork`-without-`exec` pattern is common enough to call out on its own: the trick a long-running service uses to detach from its launcher and keep running in the background — what Unix calls "daemonizing," the `fork`-then-`setsid`-then-`fork`-again dance. It depends entirely on `fork` and so has no Unix form on QSOE.

QSOE provides the operation directly instead, with a single call carried over from QRV, the predecessor project QSOE continues (QNX has no equivalent — its nearest call, `procmgr_daemon`, is a different thing):

```
int procmgr_detach(int status);
```

A resident server does its setup in `main()` as an ordinary foreground program — opens its devices, registers the path it serves — and then, once it is ready to serve, calls `procmgr_detach()`. The system server (`taskman`) reparents it to `pid 1` and the program simply keeps running; there is no second process and no `fork`.

The call does one more thing that makes it better than the idiom it replaces: it is also a **readiness handshake**. The launcher that spawned the server is parked in `waitpid()`, and `procmgr_detach` is what releases it — handing back the `status` value — so the launcher learns the service is actually *up*, not merely started. QSOE's own start-up is built on exactly this: each driver in `init` runs to the point of registering its path, calls `procmgr_detach`, and only then does the next line run, already able to see it. Porting a service therefore means replacing its self-backgrounding code with a single `procmgr_detach()` at the point it becomes ready — usually simpler than the original.

3.2 Multiplexing: `poll` instead of `select`

QSOE has no `select()`; `poll()` is the one and only I/O multiplexer, permanently. Translating `select` to `poll` is the standard, mechanical rewrite — build a `struct pollfd` array instead of the three `fd_set`s, and read `revents` instead of testing the sets:

```
struct pollfd pfd[2];
pfd[0].fd = sock;  pfd[0].events = POLLIN;
pfd[1].fd = tty;   pfd[1].events = POLLIN;
int n = poll(pfd, 2, timeout_ms);
if (pfd[0].revents & POLLIN) { /* sock readable */ }
```

Planned content. There is a catch specific to 0.1, and it is a library gap layered on top of the permanent structural choice: `poll()` itself is not yet fully implemented. In 0.1 it reports readiness immediately rather than blocking on real I/O readiness — enough for code paths that poll-then-read, but not a true readiness wait. The real readiness layer is planned. So translate

`select` to `poll` now (that is the permanent shape), but do not yet rely on `poll` to block waiting for an event.

3.3 Low-level memory: `mmap` instead of `brk`

QSOE has no `brk()` or `sbrk()`. Low-level memory comes only through mmap-based calls, and `mmap`, `munmap`, and `mprotect` are provided.

For almost all code this is invisible, because the C heap works normally: `malloc`, `calloc`, `realloc`, and `free` are all present and are themselves built on `mmap`, not on `brk`. The only code that needs attention is the rare program that calls `sbrk()` *directly* — some custom allocators do — which must be rebuilt on `mmap`. A program that merely uses `malloc` needs no change at all.

3.4 No copy-on-write

Copy-on-write is a hard non-goal: without `fork()` there is nothing for it to optimize, so QSOE does not implement it. Every spawned process gets fresh frames; nothing of the parent's memory is shared into the child implicitly.

This matters to a port only where code assumed the cheap, implicit sharing that `fork` gives — for instance, forking after building a large in-memory table so each child "inherits" it for free. On QSOE that inheritance does not happen, so the data must be passed deliberately: built in each process, sent as a message, or placed in memory that was explicitly arranged to be shared. Sharing on QSOE is always something you ask for, never something you get by accident — which is the same principle as the absence of `fork` itself, seen from the memory side.

3.5 Signals arrive as pulses (but look normal)

QSOE delivers signals as *pulses* — short fixed messages — rather than through the kernel-resident mechanism a monolithic Unix uses. For a port this is mostly an implementation detail you do not see: the POSIX surface is the ordinary one. `signal` and `sigaction` register handlers, `kill` and `raise` send signals, and `sigprocmask` masks them, all as usual. Every program runs a small system thread from the start that receives these pulses and runs your handlers, so a handler installed with `sigaction` fires the way you expect, including for a signal sent by another process.

Two points are worth knowing as you port:

- `sigaction` is process-local — a disposition lives in your program's own C library, and the system server never sees it. This is invisible to correct POSIX code; it only matters if you assumed something about where dispositions are held.
- Signaling a whole process group (`kill` with a non-positive pid) works as of 0.2, and is what carries Ctrl-C to a pipeline. It takes a different route from a single-pid signal: only the process manager can enumerate a group, so the walk and the pulses happen there, in one request.

Chapter 4

The libc Surface and Its Gaps

The structural differences of the last chapter are few and permanent. The larger source of porting friction is simpler and more fluid: the C library is young, and many standard functions are not written yet. These are not design exclusions — they are work not yet done, and the list shrinks with every release. 0.2 alone added a little over a hundred functions, which is why the *C Library Reference* (QSOE-LCR) exists as a delta rather than as a complete reference. This chapter is about meeting that reality productively: how a gap shows itself, what is missing by area, and what to do about each one.

4.1 How a gap shows itself

Because QSOE programs compile against QSOE's own headers only (§2.2), a missing function cannot hide. It surfaces in one of two honest ways:

- **At compile time** — if no QSOE header declares the function, the reference does not compile. This is the common case, and the best one: you learn of the gap before you have a binary.
- **At load time** — if a function is declared but not yet implemented in `libc.so`, the program links but the loader reports the unresolved symbol *by name* when you run it, because binding is eager (§2.3). Either way the missing name is handed to you precisely.

There is no third case where a gap lurks until some rare path runs: QSOE will not silently substitute a host function, because the host's headers and libraries are not in the build at all. A port that builds and loads has, by construction, every library function it references.

To check *before* you start — to gauge a port's feasibility rather than discover it function by function — consult the inventory of missing functions in Appendix A, which lists, per category, exactly what the current release does and does not provide.

4.2 What is missing, by area

The gaps are not spread evenly; they cluster. Knowing the clusters lets you judge a port quickly — code that lives in a well-covered area ports easily, while code that leans on a thin one needs more thought. The picture in 0.2, measured against the built library rather than remembered:

- **Core C is largely there.** The everyday C library — `printf` and the stdio streams, `malloc`, the `str*` and `mem*` functions, `ctype`, the common POSIX file and process calls — is mostly present, which is why a simple program tends to build and run.

- **Filesystem mutation arrived with somewhere to write.** This was the largest gap in 0.1 and is mostly closed: `mkdir`, `rmdir`, `unlink`, `chmod`, `chown`, `truncate` and `ftruncate` are all present, because 0.2 has a writable filesystem for them to act on (§5.1). `rename` links but is an announcing stub that always fails with `EROFS`: an atomic rename is 0.3 work, so the write-to-temporary-then-rename idiom has no equivalent yet. `link` and `mkfifo` remain absent — hard links are not something the filesystem models, and a named pipe is a resource manager nobody has written yet.

- **Terminal control is partial.** The basic `tcgetattr`/`tcsetattr` path exists, and `tcdrain` joined it, but the rest of `termios` — `tcflush`, `tcflow`, and the `cf*` speed helpers — does not. Code doing heavy terminal manipulation will meet gaps. Note also that the console has no line discipline of its own: `termios` is honored by the library, so a program that sets raw mode and cooks its own input behaves as expected, while one that assumes the driver echoes for it does not.

- **Real-time POSIX is thin.** Asynchronous I/O (`aio_*`), message queues (`mq_*`), most of the semaphore and scheduling APIs, and the advanced synchronization of POSIX.1j (barriers, spinlocks) are largely unimplemented.

- **Threads: the core is there, the periphery is thinner.** Thread creation, mutexes and condition variables work, and some attribute setters (`pthread_attr_setstacksize` among them) have landed. Read/write locks and the cancellation machinery (`pthread_cancel`) have not — and cancellation is a design question rather than a missing file: killing a thread that never blocks is unsolved on both kernels.

- **Wide characters and locales are minimal.** The narrow, C-locale path is well covered; the wide-character streams and string functions (`wprintf`, `wcs*`) and the locale framework are mostly absent, though 0.2 added the three restartable UTF-8 conversions (`mbrtowc`, `wcrtomb`, `mbsinit`). There is one locale, `C.UTF-8`, and no locale framework to change it, so software that is i18n-heavy is not a good fit for QSOE at all rather than merely for this release.

- **Time conversion is essentially complete.** Another 0.1 gap that closed: `gmtime`, `localtime`, `mktime`, `ctime`, `asctime` and `difftime` joined `clock_gettime` and `strftime`, taking the bucket to 85%. A program doing calendar arithmetic no longer needs its own.

- **There are no sockets.** Neither the socket calls nor the address-conversion helpers (`inet_pton`, `inet_ntop`) exist, and software written against BSD sockets does not port.

  This is not the same as saying QSOE has no networking. 0.2 has QSP (the Networking Guide, `QSOE-NET`): one machine opens a path on another and the resource manager at the far end is never told anything happened. A distributed QSOE program is written with `open` and `read`, not with a socket — so the porting question is not "where are the sockets" but whether the program's distribution can be expressed as somebody else's files.

Appendix A gives the exact, per-category lists behind this summary. They are generated from the built library rather than written by hand — a function counts as present only if `libc.so` exports it — so the appendix cannot drift from the tree the way a remembered list does.

4.3 Coping with a gap

When your program needs something QSOE does not have, there are four responses, roughly in order of preference:

1. **Avoid it.** Many programs reach a missing function only through an optional feature or a configuration default. Disabling that feature, or choosing a code path that does not need the call, is often the whole fix — and costs QSOE nothing.

2. **Provide it yourself.** A gap that is genuinely a small, kernel-independent function (a string helper, a calendar-time conversion) can be carried inside your own program, compiled from portable source you bring with you. This keeps your port building today without waiting on the library, and it touches nothing outside your component.

3. **Wait for it.** If the function is squarely in libc's remit and simply not written yet, it is on the path to being implemented; for software that is not urgent, the gap may close on its own in a later release. Appendix A is, from the other side, exactly that worklist.

4. **Contribute it.** QSOE is Apache-2.0 and the library is meant to grow. A function you implement cleanly — in the right place in the shared C library, against a portable reference — closes the gap for everyone, and turns a port that revealed a hole into one that filled it.

The right choice depends on the function and on your goal. A one-off port favors avoiding or self-providing; work meant to land in QSOE for good favors contributing. What you should *not* do is assume a gap is permanent: with the exception of the deliberate structural choices of Chapter 3, a missing function is a function not yet written, not a function refused.

Chapter 5

The Runtime Environment

A program that builds, links, and loads has cleared the compile-time and link-time hurdles — but it then runs in an environment that differs from a mature Unix in a few ways that matter at *run* time. This chapter covers those: the read-only filesystem of 0.1, the namespace a program sees, the environment it inherits, and the console it talks to. None of these is a build problem; each is something a running port can trip over if it assumes the Unix default.

5.1 Where a program may write

The most important runtime fact about 0.1 was that nothing could be written anywhere. That is no longer true, and it is the single change that most widens what will port.

The shape to hold in mind is that **the root filesystem is still read-only, and writable space is mounted onto it**. `fs-tmpfs` provides that space, and the boot mounts it at the places a working session needs:

- `/tmp` — scratch, mounted early because `fs-tmpfs` depends on nothing: no disk, no block driver, no root filesystem. It is therefore available even to a rescue shell.
- `/var` — for a program that wants to write state.
- Writable directories under the user's home.

So the calls work: `O_CREAT`, `mkdir`, `unlink`, `rename` and `ftruncate` all reach a filesystem that implements them (§4.2). A lock file, a cache, a `/tmp` working file, a dotfile — the things Unix software writes as a matter of course — have somewhere to go.

Two caveats remain, and both are about *where* rather than whether:

- **It is memory, not a disk.** `fs-tmpfs` keeps its store in RAM, so a mount does not survive a reboot and its size is a cap on what can be written into it. A program that expects its output to still be there next boot needs a path on the real filesystem, and 0.2 has no writable on-disk filesystem to offer it.
- **Writing outside a mount still fails.** A program that insists on writing next to its own binary, or anywhere else on the read-only root, fails. The fix is configuration rather than code: point it at `/tmp` or `/var`.

A partial truncation — shrinking a file to a smaller non-zero size — is refused with `EINVAL`; truncating to zero and removing a file both work.

Planned content. A writable in-memory filesystem (`ramfs`) is a required part of 0.2, which will give programs real scratch space — ordinary read/write files and directories. Much write-dependent software that cannot run on 0.1 will simply work once it lands.

5.2 The namespace a program sees

QSOE presents a single pathname space, and for a port it is reassuringly conventional. Familiar paths are where you expect: `/dev` for devices, `/etc` for configuration (it is a symbolic link into the system configuration area, so `/etc/passwd` and friends resolve normally), `/home` for home directories, and the program's own data under `/usr`.

Two QSOE traits are worth knowing, though neither usually requires action. First, the tree is assembled from several servers rather than one filesystem — `/dev`, `/sys`, and `/proc` are served by drivers and the system server, not stored on disk — but a program reaches them through ordinary `open`/`read`/`write`, so this is invisible to portable code. Second, those same calls work on devices because a device is just a path served by its driver; code that opens `/dev/...` and reads or writes it behaves as it would on Unix.

5.3 The inherited environment

A program started from the shell inherits a conventional environment. Login sets the usual variables — `HOME`, `PATH`, `USER`, `LOGNAME`, `SHELL`, and `PWD` — and `getenv` and `setenv` work normally, as does `environ`.

One omission is worth calling out because it catches interactive software: **`TERM` is not set**. A program that consults `TERM` to drive a terminal database (the `curses`/`terminfo` family) will not find it, which is consistent with the console limits below — full-screen terminal software is not supported on 0.1 in any case. Plain line-oriented programs are unaffected.

5.4 The console and terminals

The system console may be a serial line or, on a machine whose firmware hands over a text-mode video controller, the screen with a USB keyboard on it — 0.2 added the second and a program cannot tell them apart through the C library. Either way terminal handling is deliberately minimal. The `termios` calls exist as an interface — `tcgetattr`, `tcsetattr`, `isatty` are present — but the console does *not* provide a full kernel terminal with the usual line-discipline behavior. In practice this means two things for a port:

- **Terminal-mode control is limited.** Setting raw or cbreak mode through `tcsetattr` does not have the full effect it would on a Unix tty; a program that relies on flipping terminal modes for character-at-a-time input cannot assume the usual result yet.
- **There is no full-screen terminal layer.** With no `TERM` (§5.3) and no complete terminal semantics, `curses`-style full-screen programs are out of scope for 0.1. Line-oriented interactive programs — read a line, print a response — are the supported shape, and work well.

This is the same reality the shell's own line editor reflects (the *User Guide* describes it): cooked input and editing on QSOE are done in userland, by the program or the library, not by a kernel line discipline. Software written for line-at-a-time interaction ports cleanly; software that drives the terminal as a full-screen device should wait for the terminal support that later releases will bring.

5.5 In short

A ported program on 0.1 can rely on the conventional things: the POSIX process model, signals, a normal environment and pathname space, dynamic linking, and line-oriented console I/O.

What it cannot yet assume is the ability to write the filesystem, full terminal control, a set `TERM`, or networking. A program that reads, computes, and writes line-oriented output to the console is in the sweet spot of what 0.1 runs well — and that covers a great deal of classic Unix tooling.

Chapter 6

Porting in Practice

The preceding chapters cover the pieces; this one puts them together into a workflow, and then walks a small, representative port from start to finish. The aim is to make the process concrete — to show that for software in QSOE's sweet spot, a port is short and orderly, not a leap of faith.

6.1 A porting checklist

A port proceeds in roughly this order. Each step leans on an earlier chapter, so the checklist doubles as a map back into the guide.

1. **Assess the dependencies.** Skim what the program uses. Does it rely on a permanent structural choice — `fork` without `exec`, `select`, raw `sbrk` (Chapter 3)? Does it lean on an area the library is thin in — networking, wide characters, real-time POSIX, filesystem writes (Chapter 4, Appendix A)? This first read tells you whether the port is an afternoon or a project.

2. **Set it up as a component.** Put the sources in a `quser` component with a short Makefile, so the build applies the right toolchain, headers, and link contract for you (§2.4).

3. **Build, and read the errors as a worklist.** Because the build sees only QSOE's headers, the compile errors *are* the list of missing pieces (§4.1). Work through them; each is a structural difference to adapt or a library gap to handle.

4. **Adapt the structural differences.** Apply the mechanical substitutions of Chapter 3: `posix_spawn` for `fork`, `poll` for `select`, and so on.

5. **Handle the library gaps.** For each missing function, choose from the four responses of §4.3: avoid it, provide it yourself, wait, or contribute it.

6. **Run against the 0.1 runtime.** Once it links and loads, check it against the runtime realities of Chapter 5 — above all, that it does not depend on writing the filesystem.

6.2 A worked example: a line-counting filter

Consider porting a small Unix filter — a utility that reads standard input, counts its lines, words, and characters, and writes the totals to standard output, with a few command-line flags to select which counts to show. It is a deliberately ordinary program, and a good representative of what ports easily.

Step 1 — Assess. What does it use? Standard I/O (`getchar`, `printf`), character classification (`isspace`), and option parsing. None of that touches a structural difference: it does not `fork`, multiplex, or manage memory directly. It reads standard input and writes standard output — it never writes the filesystem — so it sits exactly in the 0.1 sweet spot (§5.5). The one thing to check is the option parsing: the original uses `getopt_long` for GNU-style `--lines` long options.

Step 2 — Set it up. The program is one source file, so the component Makefile is the minimal one from §2.4:

```
include ../../common.mk
PROGRAM := wcount
SRCS    := wcount.c
include $(QUSER)/build_prog.mk
```

Step 3 — Build, and read the errors. The first build fails, and tells us exactly why:

```
wcount.c: error: implicit declaration of function 'getopt_long'
```

Everything else compiled: `getchar`, `printf`, and `isspace` are all present, so the stdio and ctype parts gave no trouble. The single gap is `getopt_long`, which 0.1 does not provide — there is no `getopt.h` declaring it (§4.1).

Step 4 — Structural differences. There are none here; nothing in the program relies on a removed facility. This is the common case for a filter.

Step 5 — Handle the gap. QSOE *does* provide POSIX `getopt` (the short-option parser); only the GNU long-option extension is missing. Since this program's long options are just aliases for its short ones, the smallest fix is to **avoid** the gap (§4.3, response one): switch the parsing loop from `getopt_long` to `getopt` and accept short options only.

```
/* before: GNU long options */
int c = getopt_long(argc, argv, "lwc", longopts, NULL);

/* after: POSIX getopt, short options only */
int c = getopt(argc, argv, "lwc");
```

If the long options had to stay — because scripts depend on them — the next response would apply instead: **provide** a small portable `getopt_long` from your own source. Here, avoidance is enough.

Step 6 — Run. With that one change the program builds, links, and loads. It is a filter, so the read-only filesystem is no obstacle: it reads standard input and writes standard output.

```
[/home/user]$ echo "porting is fun" | wcount
      1       3      15
```

The port is done. The whole of it was one mechanical substitution, found for us by the compiler and resolved by the simplest of the four strategies. That is the experience to expect for software in QSOE's range: read-oriented, line-oriented, standard-C programs port quickly, and the build itself tells you the short list of what to fix.

6.3 When a port is bigger than this

Not every port is an afternoon. A program that wants to write files, drive a full-screen terminal, open a socket, or fork a worker pool is asking for things 0.1 does not have, and the assessment step (§6.1) should surface that before you begin. For such software the honest options are to wait for the release that closes the gap — the writable filesystem of 0.2, terminal and networking support later — or to take part in building it (§4.3). The structural differences aside, nothing here is a wall; it is a matter of which release a given program's needs are met in.

Appendix A

Missing libc Functions in QSOE 0.2

This appendix lists the standard-library functions that QSOE 0.2 does *not* yet provide. It is regenerated from the built library at each release — see the note at the head of `libc-gaps.tex` — so the counts describe the release named in them and not a remembered figure. It is the exact, per-category companion to the summary in Chapter 4: a feasibility reference you can scan before a port, and a worklist of what the library has still to grow.

What this list is, and how to read it

A function is counted **present** if the built `libc.so` actually exports a symbol of that name — the same honest, link-time signal a port itself sees (§4.1). Everything else in the standard surface is **missing** and listed below. A few present functions are only *stubs* (Chapter 4); those count as present here, since they link.

A few things are deliberately *not* in this list:

- **The design exclusions** — `fork` and the `exec*` family, `select`, `brk`, and their kin. Those are permanent (Chapter 3), not gaps, so they do not belong on a list of things still to come.
- **QSOE's own native API** — message passing, the resource-manager framework, the `Interrupt*`/`Sync*`/`Timer*` calls. That surface is QSOE's core, not portable third-party code, and is documented in the Programming Book.
- **The UNIX/BSD legacy-compatibility routines** — the old `<err.h>` helpers, the pre-POSIX signal calls (`sigsetmask` and kin), `getwd`, `re_comp`, `utmp` accounting, and the rest. These are out of scope on purpose: the BSD routines real code actually relies on (`strlcpy`, `strlcat`, `memmove`, the `str*`/`mem*` family, `getopt`) are present, counted under the C and string categories; what remains is deprecated aliases with a standard replacement QSOE already provides, plus system-administration and login-accounting calls an ordinary program never makes. A clean, modern C library is right not to chase them, so a near-empty score there would be misleading rather than informative.

This appendix therefore covers the standard C and POSIX surface a port draws on.

The figures are **machine-generated from the built library** and regenerated for each release, so they track the real system rather than a hand-kept list that could drift. Treat the whole appendix as a snapshot of the release it names: every later one moves functions from the missing column to the present one, and the lists here shrink accordingly. Between 0.1 and 0.2 they shrank by a little over a hundred entries.

Coverage by category

The standard surface stands at the coverage below. The categories follow the POSIX and library groupings; the per-category function lists follow.

Category	Present	Missing	Coverage
POSIX.1	78	49	57%
POSIX.1a	8	3	73%
POSIX.1b RT	5	56	8%
POSIX.1c threads	22	44	33%
POSIX.1d RT	8	46	15%
POSIX.1g net	0	4	0%
POSIX.1j RT	1	38	3%
Allocator	5	21	18%
C89 / locale / ctype	49	18	73%
stdio	42	43	49%
string	41	55	43%
time	11	2	85%
X/Open	11	62	15%
Total	**281**	**441**	**38%**

POSIX.1 base — process / files / signals / terminal

49 missing in 0.2:
`_sysconf`, `_umask`, `_vfcntl`, `cfgetispeed`, `cfgetospeed`, `cfsetispeed`, `cfsetospeed`, `ctermid`, `dircntl`, `eaccess`, `fdatasync`, `fgetspent`, `flink`, `fsync`, `futime`, `getgrgid_r`, `getgrnam_r`, `getgroups`, `getlogin`, `getlogin_r`, `getpwent_r`, `getpwnam_r`, `getpwuid_r`, `getsid`, `getspent_r`, `getspnam_r`, `link`, `lseek64`, `mkfifo`, `mknod`, `pathconf`, `prt_grent`, `prt_pwent`, `putpwent`, `putspent`, `readdir64`, `readdir_r`, `setgroups`, `setpgid`, `sigpending`, `sigsuspend`, `tcdropline`, `tcflow`, `tcflush`, `tcsendbreak`, `tell`, `tell64`, `ttyname`, `utime`

POSIX.1a — path/regex/glob/system/wordexp

3 missing in 0.2:
`symlink`, `wordexp`, `wordfree`

POSIX.1b realtime — aio, mmap, mq, sem, sched, sig*wait, timer

56 missing in 0.2:
`_aio_clean_up`, `_aio_destroy`, `_aio_init`, `_aio_insert`, `_aio_insert_prio`, `_nsem_close`, `_nsem_open`, `_timer_round_interval`, `_vmq_open`, `aio_cancel`, `aio_error`, `aio_fsync`, `aio_read`, `aio_read64`, `aio_return`, `aio_suspend`, `aio_write`, `aio_write64`, `lio_listio`, `lio_listio64`, `mlock`, `mlockall`, `mphys`, `mq_getattr`, `mq_notify`, `mq_open`, `mq_receive`, `mq_send`, `mq_setattr`, `mq_unlink`, `msync`, `munlock`, `munlockall`, `sched_get_priority_max`, `sched_get_priority_min`, `sched_getparam`, `sched_getscheduler`, `sched_rr_get_interval`, `sched_setparam`, `sched_setscheduler`, `sem_close`, `sem_getvalue`, `sem_open`, `sem_unlink`, `shm_close`, `shm_open`, `shm_unlink`, `sigqueue`, `sigtimedwait`, `sigwait`, `sigwaitinfo`, `timer_create`, `timer_delete`, `timer_getoverrun`, `timer_gettime`, `timer_settime`

POSIX.1c threads — pthread_* attributes, rwlocks, cancellation

44 missing in 0.2:
`pthread_atfork`, `pthread_attr_getdetachstate`, `pthread_attr_getinheritsched`, `pthread_attr_getschedparam`, `pthread_attr_getschedpolicy`, `pthread_attr_getscope`, `pthread_attr_getstack`, `pthread_attr_getstackaddr`, `pthread_attr_getstacklazy`, `pthread_attr_getstackprealloc`, `pthread_attr_getstacksize`, `pthread_attr_setinheritsched`, `pthread_attr_setschedparam`, `pthread_attr_setschedpolicy`,

pthread_attr_setscope, pthread_attr_setstack, pthread_attr_setstackaddr,
pthread_attr_setstacklazy, pthread_attr_setstackprealloc, pthread_cancel,
pthread_condattr_getpshared, pthread_condattr_setpshared, pthread_getschedparam,
pthread_kill, pthread_mutex_getprioceiling, pthread_mutex_setprioceiling,
pthread_mutexattr_getprioceiling, pthread_mutexattr_getprotocol,
pthread_mutexattr_getpshared, pthread_mutexattr_getrecursive,
pthread_mutexattr_gettype, pthread_mutexattr_setprioceiling,
pthread_mutexattr_setprotocol, pthread_mutexattr_setpshared,
pthread_mutexattr_setrecursive, pthread_mutexattr_settype, pthread_rwlock_destroy,
pthread_rwlock_init, pthread_rwlockattr_destroy, pthread_rwlockattr_getpshared,
pthread_rwlockattr_init, pthread_rwlockattr_setpshared, pthread_setschedparam,
ttyname_r

POSIX.1d additional realtime — posix_spawn, fadvise, timedlock

46 missing in 0.2:
_posix_spawnattr_t_realloc, file_open_actions_fixup, get_attrp, get_factp,
mq_timedreceive, mq_timedreceive_monotonic, mq_timedreceive_woption, mq_timedsend,
mq_timedsend_monotonic, mq_timedsend_woption, posix_fadvise, posix_fadvise64,
posix_fallocate, posix_fallocate64, posix_spawn_file_actions_endswap,
posix_spawn_file_getactions, posix_spawnattr_addpartid, posix_spawnattr_addpartition,
posix_spawnattr_destroy, posix_spawnattr_endswap, posix_spawnattr_getnode,
posix_spawnattr_getpartid, posix_spawnattr_getrunmask, posix_spawnattr_getschedparam,
posix_spawnattr_getschedpolicy, posix_spawnattr_getsigdefault,
posix_spawnattr_getsigignore, posix_spawnattr_getsigmask, posix_spawnattr_getstackmax,
posix_spawnattr_getxflags, posix_spawnattr_setflags, posix_spawnattr_setnode,
posix_spawnattr_setpgroup, posix_spawnattr_setrunmask, posix_spawnattr_setschedparam,
posix_spawnattr_setschedpolicy, posix_spawnattr_setsigdefault,
posix_spawnattr_setsigignore, posix_spawnattr_setsigmask, posix_spawnattr_setstackmax,
posix_spawnattr_setxflags, posix_spawnattr_t_once_init, set_attrp, set_factp,
valid_attrp, valid_factp

POSIX.1g networking — inet_pton/ntop (no sockets)

4 missing in 0.2:
htonl, htons, ntohl, ntohs

POSIX.1j advanced realtime — barriers, spinlocks, clock_nanosleep

38 missing in 0.2:
_pthread_spin_destroy, _pthread_spin_init, _pthread_spin_lock, _pthread_spin_trylock,
_pthread_spin_unlock, _spin_destroy, _spin_init, barrier_attr_destroy,
barrier_attr_getpshared, barrier_attr_init, barrier_attr_setpshared, barrier_destroy,
barrier_init, barrier_wait, clock_nanosleep, mem_access_clear, mem_access_set,
mem_get_info, pthread_abort, pthread_barrier_destroy, pthread_barrier_init,
pthread_barrier_wait, pthread_barrierattr_destroy, pthread_barrierattr_getpshared,
pthread_barrierattr_init, pthread_barrierattr_setpshared, pthread_condattr_getclock,
pthread_spin_destroy, pthread_spin_init, pthread_spin_lock, pthread_spin_trylock,
pthread_spin_unlock, spin_destroy, spin_init, spin_lock, spin_trylock, spin_unlock,
timer_getexpstatus

memory allocator

21 missing in 0.2:
_band_get, _band_get_aligned, _check_bands, _flist_bin_first_fit, _heap_alloc, _msize,
_posix_memalign, _salloc_init, _scalloc, _sfree, _smalloc, _srealloc, _sreallocfunc,
crash, data_check, data_set, malloc_pc, memalign_pc, memory, qnx_heap_alloc, tstcheck

C89 / locale / wide chars / ctype

18 missing in 0.2:
`_findenv, _tolower, _toupper, atexit, btowc, isascii, lldiv, localeconv, mblen, mbrlen, mbsrtowcs, mbstowcs, rand_r, remove, setlocale, towctrans, wctob, wctrans`

stdio — regular and wide-char streams, scanf/printf

43 missing in 0.2:
`_freopen, _fsopen, _tmpfile, fcloseall, fgetchar, fgetwc, fgetws, flockfile, flushall, fopen64, fputchar, fputwc, fputws, freopen, freopen64, fseeko64, ftello64, ftrylockfile, funlockfile, fwide, fwprintf, fwscanf, getc, getchar, gets, getwc, getwchar, putwc, putwchar, setbuf, swprintf, swscanf, tmpfile, tmpfile64, ungetwc, vfwprintf, vfwscanf, vswprintf, vswscanf, vwprintf, vwscanf, wprintf, wscanf`

string and wide-string handling, numeric parsers

55 missing in 0.2:
`_memicmp, _strdup, _strlwr, _strupr, atoh, bcmp, index, itoa, lltoa, ltoa, memcpyv, memicmp, rindex, straddstr, strcoll, strlwr, strnset, strrev, strset, strtok_r, strupr, strxfrm, ulltoa, ultoa, utoa, wcscat, wcscmp, wcscoll, wcscpy, wcscspn, wcsncat, wcsncmp, wcsncpy, wcspbrk, wcsrchr, wcsrtombs, wcsspn, wcsstr, wcstod, wcstof, wcstoimax, wcstok, wcstol, wcstold, wcstoll, wcstombs, wcstoul, wcstoull, wcstoumax, wcsxfrm, wmemchr, wmemcmp, wmemcpy, wmemmove, wmemset`

time conversion / formatting / DST

2 missing in 0.2:
`clock, wcsftime`

X/Open extensions — statvfs, lockf, drand48, rlimits, etc.

62 missing in 0.2:
`FinalTr, Indication, InitialTr, _unsetenv, creat64, encrypt, fchdir, fstat64, fstatvfs, fstatvfs64, ftok, ftruncate64, ftw, ftw64, getitimer, getpgid, getpriority, getrlimit, getrlimit64, getrusage, grantpt, ioctl, killpg, lfind, lockf, lockf64, lsearch, lstat64, nftw, nftw64, nice, open64, posix_openpt, pread64, pthread_attr_getguardsize, pthread_attr_setguardsize, pthread_getconcurrency, pthread_setconcurrency, ptsname, ptsname_r, pwrite, pwrite64, readv, realpath, setkey, setpgrp, setpriority, setrlimit, setrlimit64, stat64, statvfs, statvfs64, swap, sync, tcgetsid, tempnam, tmpnam, truncate64, ualarm, unlockpt, utimes, waitid`